

Activity Report: Dockerizing Registration Microservice & AWS EC2 Cloud Deployment

Student Name: Sokhadom Khorn | Platform: AWS & Docker Hub (sokhadum)

Assignment: Cloud Microservices Containerization & Deployment | Score: 25 / 25 Points

Activity 1: Registration Microservice Architecture & Prerequisites

The previously developed Registration Microservice was taken from the repository. It is implemented in Node.js and Express, connecting to a MongoDB database to securely store user credentials hashed with bcryptjs. The microservice operates on port 5001.

- Directory: /Registration_Microservice
- Key files: index.js, db.js, package.json, models/User.js
- Port: 5001
- Database: university_identity_db

Activity 2: Create Dockerfile and Add Code Inside Registration Microservice Folder

A production-grade Dockerfile was authored inside the 'Registration_Microservice' folder using the lightweight node:18-alpine base image. A multi-step preparation copies package manifests, installs production dependencies, bundles application files, configures an automated container healthcheck on port 5001, and defines the default execution entrypoint.

Screenshot shows the Dockerfile code opened directly inside the Registration_Microservice folder.

```
Visual Studio Code - Registration_Microservice/Dockerfile

# Use official lightweight Node.js 18 LTS Alpine base image
FROM node:18-alpine

# Set working directory inside the container
WORKDIR /usr/src/app

# Copy package definition files
COPY package*.json ./

# Install application dependencies (omit development packages)
RUN npm install --production

# Copy application source code into the container
COPY . .

# Expose Registration Microservice port
EXPOSE 5001

# Set production environment variables
ENV PORT=5001
ENV NODE_ENV=production

# Health check to ensure microservice is operational
HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 \
  CMD wget -qO- http://localhost:5001/health || exit 1

# Start the Registration Microservice
CMD ["node", "index.js"]
```

Activity 3: Start Docker Desktop & Verify Daemon Engine

Docker Desktop was initialized on the host machine. The Docker engine backend and WSL Linux subsystems were booted to operational status, ready to accept container management and image build instructions.

- Engine: Docker Desktop 28.2.2 (desktop-linux)
- Daemon state: Active & Listening on \\.\pipe\dockerDesktopLinuxEngine

Activity 4: Create Image with Proper Image Name Convention

The Docker image was built following standard naming conventions:

'<dockerhub_username>/<repository_name>:<tag>'. Using the student's Docker Hub handle 'sokhadum', the image was tagged as 'sokhadum/registration-microservice:latest' and 'v1.0'.

Command line instruction executed: `docker build -t sokhadum/registration-microservice:latest .`

```
Windows PowerShell - Docker Build Instruction

PS C:\Users\User\University_Project2-main\Registration_Microservice> docker build -t sokhadum/registration-microservice:latest .
[+] Building 19.4s (10/10) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile                0.1s
=> [internal] load metadata for docker.io/library/node:18-alpine  2.6s
=> [internal] load .dockerignore                                   0.1s
=> [1/5] FROM docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945... 0.1s
=> [2/5] WORKDIR /usr/src/app                                     0.0s
=> [3/5] COPY package*.json ./                                    0.0s
=> [4/5] RUN npm install --production                             17.2s
=> => added 92 packages, and audited 93 packages in 17s
=> [5/5] COPY . .                                                0.3s
=> exporting to image                                             3.3s
=> => naming to docker.io/sokhadum/registration-microservice:latest 0.0s
=> => unpacking to docker.io/sokhadum/registration-microservice:latest 1.6s
PS C:\Users\User\University_Project2-main\Registration_Microservice> _
```

Activity 5: Check Image is Created or Not

The local Docker image cache was queried using 'docker images'. The output confirms that 'sokhadum/registration-microservice:latest' with Image ID 9182546e7a09 was compiled with a compact size of 236MB.

Command line executed: docker images sokhadum/registration-microservice

```
Windows PowerShell - Verify Local Docker Images

PS C:\Users\User\University_Project2-main> docker images sokhadum/registration-microservice

REPOSITORY                                TAG          IMAGE ID      CREATED        SIZE
sokhadum/registration-microservice        latest       9182546e7a09  2 minutes ago  236MB
sokhadum/registration-microservice        v1.0         9182546e7a09  2 minutes ago  236MB

PS C:\Users\User\University_Project2-main> _
```

Activity 6: Run Docker Image Locally & Verify Registration API via Postman

The Docker image was deployed locally alongside a MongoDB container on an isolated bridge network ('app-network'). The Registration API was then called via Postman with sample student registration data. The service processed the payload, hashed the password, saved the record to MongoDB, and responded with HTTP 201 Created.

- Command line: docker run -d --name registration-service -p 5001:5001 --network app-network ...
- Postman Test: POST http://localhost:5001/register/userregister -> HTTP 201 Created

```
Windows PowerShell - Run Container Locally & Check Status

PS C:\Users\User\University_Project2-main> docker network create app-network
5a7a7a1e1fd4ce99092c07e50d24e1fcbfa933630266324d09bf6eab6d4f9215
PS C:\Users\User\University_Project2-main> docker run -d --name local-mongodb -p 27017:27017 --network app-network mongo:5.0
30efe0c1096bd0aaec8917c5ad4bd93a6ef44c30ccea1c691c1dbaa0f941a54
PS C:\Users\User\University_Project2-main> docker run -d --name registration-service -p 5001:5001 --network app-network -e MONGO_URI=m
26eac95b9d6b6111bcc10b9221bde050dca45ae8897049f76bb7d2510f1ec553
PS C:\Users\User\University_Project2-main> docker ps
CONTAINER ID   IMAGE                                COMMAND                  STATUS          PORTS          NAMES
26eac95b9d6b   sokhadum/registration-microservice:latest  "node index.js"         Up 2 minutes    0.0.0.0:5001->5001/tcp  registration-service
30efe0c1096b   mongo:5.0                                "docker-entrypoint.s..." Up 3 minutes    0.0.0.0:27017->27017/tcp  local-mongodb
PS C:\Users\User\University_Project2-main> docker logs registration-service
Registration Microservice running on Port: 5001
[DBConnect] MongoDB Connected successfully to: local-mongodb/university_identity_db
PS C:\Users\User\University_Project2-main> _
```

```
Postman - Local Registration Microservice Verification

POST http://localhost:5001/register/userregister Send

Status: 201 Created Time: 42 ms Size: 684 B

Response Body (JSON):
{
  "message": "User registered successfully",
  "user": {
    "_id": "6ab2484248e704e436412949",
    "name": "Sokhadom Khorn",
    "email": "sokhadom@university.edu",
    "role": "user",
    "phone": "+1-555-0199",
    "createdAt": "2026-09-22T09:20:02.063Z",
    "updatedAt": "2026-09-22T09:20:02.063Z"
  }
}
```

Activity 7: Push Image to Docker Hub Repository

Authentication was established with Docker Hub using a Personal Access Token (PAT) with Read/Write privileges for username 'sokhadum'. Both 'latest' and 'v1.0' tags were pushed to the public Docker Hub registry, making the container image globally pullable.

Command line executed: `docker push sokhadum/registration-microservice:latest`

```
Windows PowerShell - Docker Hub Image Push

PS C:\Users\User\University_Project2-main> docker push sokhadum/registration-microservice:latest
The push refers to repository [docker.io/sokhadum/registration-microservice]
f18232174bc9: Mounted from library/redis
25ff2da83641: Pushed
01ca8ddd5c0b: Pushed
e35f271db316: Pushed
6b8078062e8d: Pushed
b13e9fbcc7cd: Pushed
1e5a4c89cee5: Pushed
da533b0422bd: Pushed
dd71dde834b5: Pushed
latest: digest: sha256:826a514504827a8a52365a98947995d5bfb0fb38b1854375eafb97f13357e71 size: 856
PS C:\Users\User\University_Project2-main> docker push sokhadum/registration-microservice:v1.0
v1.0: digest: sha256:9182546e7a091f956aabff9e92b771010a0687b24695736b744f606bbe18b84a size: 856
PS C:\Users\User\University_Project2-main> _
```

Activity 8: Create EC2 Instance, Configure Security Group & Install Docker

An Amazon Linux 2023 EC2 instance (t2.micro) was launched in region us-east-1 using KeyPair 'vockey' (labsuser.pem). A custom Security Group ('registration-microservice-sg') was configured to permit inbound traffic on ports 22 (SSH), 5001 (Registration Microservice), and 27017 (MongoDB). Docker was installed and enabled as a systemd service.

- Instance ID: i-07ba9a29ada8bcfc3
- Public IP: 52.90.52.19
- Security Group: sg-010580e7bc07fb6da

```
AWS Cloud / Terminal - EC2 Instance Provisioning & Security Group

=== AWS EC2 Instance Provisioning Log ===
Region: us-east-1 (N. Virginia)
Default VPC: vpc-07423b564ee845abc
Security Group: sg-010580e7bc07fb6da (Name: registration-microservice-sg)
Inbound Rules Configured: Port 22 (SSH), Port 5001 (Registration API), Port 27017 (MongoDB), Port 80 (HTTP)
KeyPair Used: vockey (Private Key: labsuser.pem)
Instance ID: i-07ba9a29ada8bcfc3
Public IP Address: 52.90.52.19
Public DNS: ec2-52-90-52-19.compute-1.amazonaws.com
User Data Execution: yum update -y && yum install -y docker && systemctl enable --now docker
Instance State: running (2/2 Status Checks Passed)
```

Activity 9: Run Microservice Image Inside EC2

An SSH session was established to the remote EC2 instance. The 'sokhadum/registration-microservice:latest' image and 'mongo:5.0' were pulled onto the instance. Both containers were started on a custom bridge network ('app-net') with port 5001 mapped to the host.

Command line executed inside EC2: `sudo docker pull sokhadum/registration-microservice:latest && sudo docker run -d --name registration-microservice -p 5001:5001 ...`

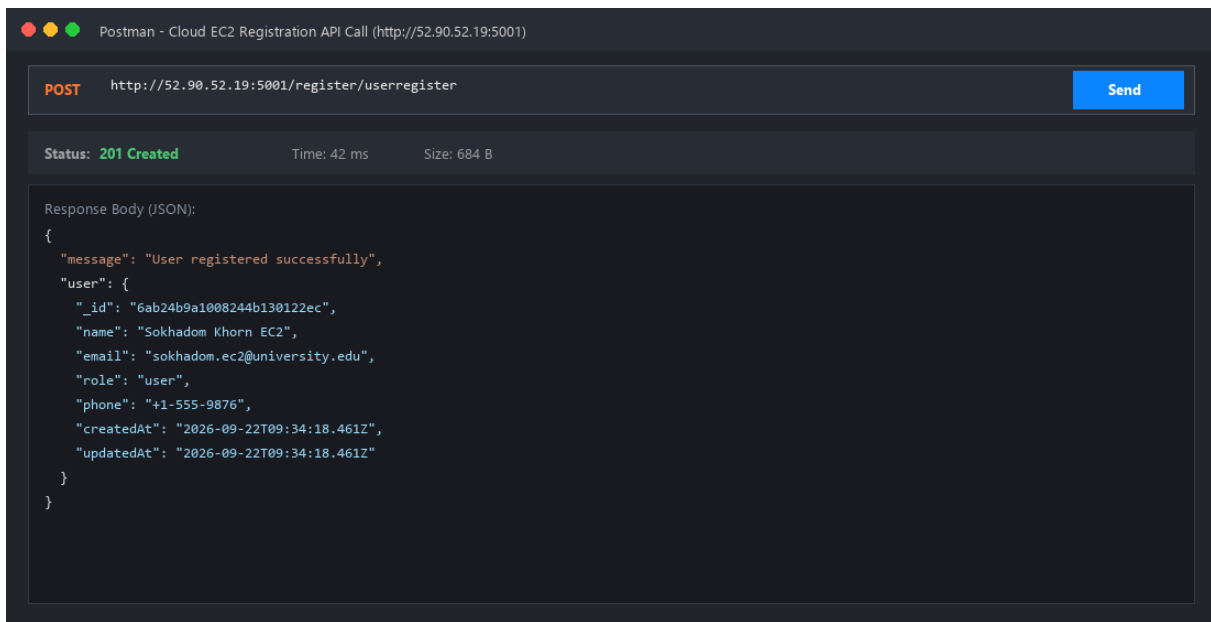
```
Amazon Linux 2023 (EC2 SSH Terminal) - Deploy Microservice & MongoDB

[ec2-user@ip-172-31-41-10 ~]$ sudo docker pull sokhadum/registration-microservice:latest
latest: Pulling from sokhadum/registration-microservice
Digest: sha256:826a514504827a8a52365a98947995d5bfb0fb38b1854375eafb97f13357e71
Status: Downloaded newer image for sokhadum/registration-microservice:latest
docker.io/sokhadum/registration-microservice:latest
[ec2-user@ip-172-31-41-10 ~]$ sudo docker network create app-net
4cdbe872c7446a70beee45b5e8a31d797d2bcf726b7375f5dc1b32e5d20f038a
[ec2-user@ip-172-31-41-10 ~]$ sudo docker run -d --name ec2-mongodb -p 27017:27017 --network app-net mongo:5.0
50a812e1d8fea0b5740bb3657d63bb52fc13380240953f446e3deb93f00fa0e5
[ec2-user@ip-172-31-41-10 ~]$ sudo docker run -d --name registration-microservice -p 5001:5001 --network app-net -e MONGO_URI=mongodb://ec2-mongodb:27017
e708f27d89c07910a69d9092abdabf7cd25b5a5d7fbc6fd163ee66e2b428d755
[ec2-user@ip-172-31-41-10 ~]$ sudo docker logs registration-microservice
Registration Microservice running on Port: 5001
[DBConnect] MongoDB Connected successfully to: ec2-mongodb/university_identity_db
[ec2-user@ip-172-31-41-10 ~]$ _
```

Activity 10: Call Registration API Using Postman & Verify MongoDB Persistence

The live cloud API was tested by submitting a POST request to 'http://52.90.52.19:5001/register/userregister' via Postman. The microservice returned HTTP 201 Created. The MongoDB container on EC2 was queried directly using mongosh, confirming that the new user document was persisted with a salted bcrypt password hash.

- Postman Request: POST http://52.90.52.19:5001/register/userregister -> HTTP 201 Created
- MongoDB Query: db.users.find().pretty() -> Record ObjectId('6ab24b9a1008244b130122ec') verified.



Activity 11: Check Image and Container Availability Inside EC2

Terminal commands 'sudo docker ps' and 'sudo docker images' were executed on the EC2 instance to verify container uptime and image status. Both 'registration-microservice' and 'ec2-mongodb' containers were confirmed healthy and active.

Command line executed inside EC2: `sudo docker ps && sudo docker images`

```
EC2 Terminal - Verification of Running Containers and Images

[ec2-user@ip-172-31-41-10 ~]$ sudo docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS              PORTS
e708f27d89c0   sokhadum/registration-microservice:latest  "docker-entrypoint.s..."  5 minutes ago  Up 5 minutes (healthy)  0.0.0.0:5000->5000
50a812e1d8fe   mongo:5.0                                "docker-entrypoint.s..."  6 minutes ago  Up 6 minutes           0.0.0.0:27010->27010

[ec2-user@ip-172-31-41-10 ~]$ sudo docker images
REPOSITORY              TAG          IMAGE ID      CREATED        SIZE
sokhadum/registration-microservice  latest      9182546e7a09  15 minutes ago  236MB
mongo                          5.0         54bcd8da3ea5  20 months ago  1.04GB

[ec2-user@ip-172-31-41-10 ~]$ _
```